**OBJECT-ORIENTED ANALYSIS AND DESIGN CAT 1**

**BSCCS/2023/65632**

**1. Three Major Types of Relationships Among Objects**

I. **Association**:

- Represents a relationship between two objects, where they interact or are linked in some way.
- Example: A "Student" and a "Course" have an association since students enroll in courses.

II. **Aggregation**:

- A special form of association that represents a whole-part relationship where the part can exist independently of the whole.
- Example: A "Library" contains "Books," but if the library is closed, the books still exist.

III. **Composition**:

- A stronger form of aggregation where the part cannot exist without the whole.
- Example: A "House" and its "Rooms" have a composition relationship. If the house is destroyed, the rooms do not exist.

**2. Purpose of Activity Diagrams and Interaction Diagrams**

- **Activity Diagram**: Shows the flow of activities or actions in a system. It focuses on the sequence of activities and is used to model workflows.
- **Interaction Diagram**: Illustrates the interactions between objects to achieve a specific task. It includes sequence diagrams and collaboration diagrams, focusing on the exchange of messages.

**3. Communication Between Objects in Object Orientation**

- Objects communicate by sending messages to each other. One object calls a method on another object, providing data (arguments) if needed.
- **Example**: In a banking system, an "Account" object can send a "withdraw()" message to an "ATM" object, instructing it to deduct a specific amount.

**4. Perspectives in Class Diagrams**

I. **Conceptual Perspective**:

- Represents real-world concepts and their relationships.
- Focuses on the problem domain without technical details.

II. **Specification Perspective**:

- Describes the interfaces and behavior of the system components.
- Focuses on the services the system provides to its users.

III. **Implementation Perspective**:

- Represents the actual implementation details of the classes and their relationships.
- Includes data types, method definitions, and programming constructs.

**5. Abstract Class and Its Importance to Inheritance**

- **Abstract Class**: A class that cannot be instantiated and is meant to be a base class for other classes. It can have abstract methods (methods without implementation) and concrete methods.
- **Importance**: It provides a blueprint for subclasses, ensuring they implement specific methods and encouraging code reusability and polymorphism.

**6. Class Diagram for Motor Dealer Order System**

✓ **Classes:**

- **Order**: Contains a list of items.
- **Item**: Represents individual parts with attributes like part-number, name-of-item, and item price.

✓ **Operations**:

- Add an item, delete an item, print total value.

The class diagram will show:

- **Order** class with operations and attributes.
- **Item** class associated with **Order**.

**7. Computerized Telephone Book Classes and Operations**

✓ **Classes**:

- **Entry**: Represents individual records in the telephone book.
- **Administrator**: Manages modifications, additions, and deletions.
- **User**: Can look up entries.

- ✓ **Operations:**

- Look up entries, add, delete, modify entries, print the telephone book.

**8. Use Case Diagram for College Library System**

- ✓ **Actors**:

- **Student** and **Academic Staff**: Can borrow, reserve, and browse items.
- **Library Counter Staff**: Registers members.
- **System Manager**: Updates the catalogue and generates reports.

Use cases:

- Borrow, reserve, return items, update catalogue.

**9. Object Interaction Diagram for Hospital Scenario**

The interaction diagram will show the sequence of activities involving:

- **Patient**, **Reception**, **Ward**, **Nurse**, **Anesthetist**, **Consultant**, **Duty Doctor**, and others.

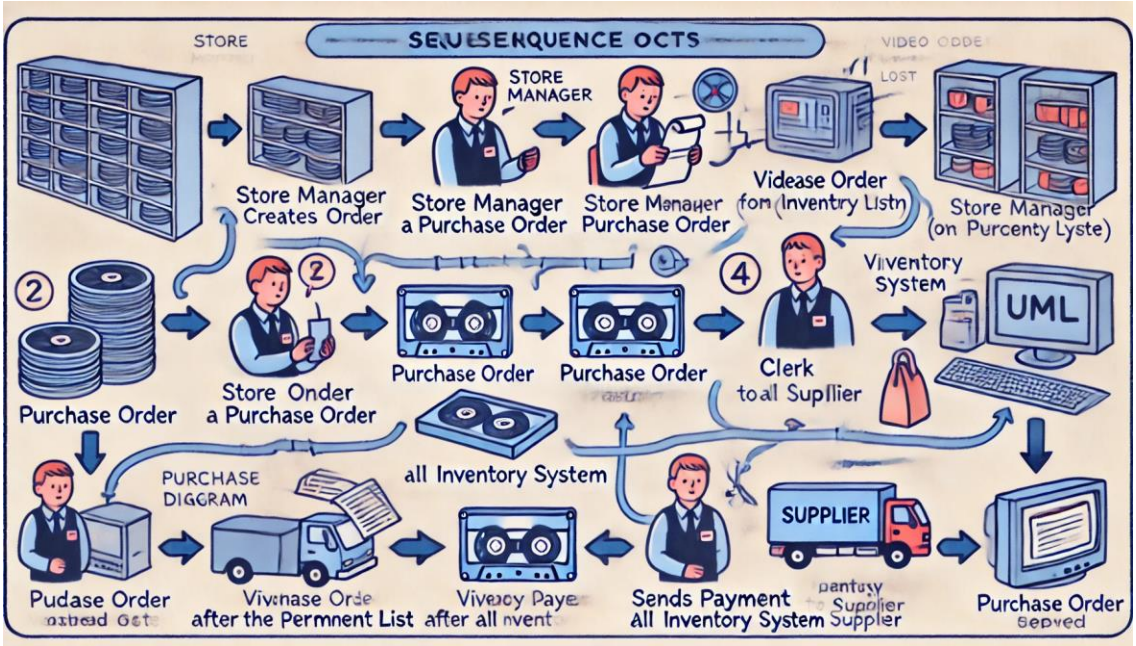
It will detail the patient's journey from reception to discharge.

**10. Sequence Diagram for Videotape Order Scenario**

The sequence diagram will illustrate:

- The **Store Manager** creating a purchase order.
- **Clerk** updating the order upon receiving videotapes.
- The final payment process once all tapes are received.

**11) Construct a sequence diagram for the scenario above**




**12. Differences Between Object-Oriented Concepts**

I. **Aggregation vs Inheritance**:

- **Aggregation**: Represents a whole-part relationship (e.g., Car and Engine).
- **Inheritance**: A hierarchical relationship where a class derives properties from another class (e.g., Dog inheriting from Animal).

II. **Composition vs Specialization**:

- **Composition**: A stronger form of aggregation where the part cannot exist without the whole (e.g., House and Room).
- **Specialization**: Creating a subclass with more specific attributes (e.g., Bird as a specialization of Animal).

III. **Generalization vs Abstraction**:

- **Generalization**: Creating a base class from common attributes of multiple classes (e.g., Vehicle for Car, Bike).
- **Abstraction**: Focusing on essential details while hiding complexity (e.g., a simplified representation of a database).